



南開大學
Nankai University

计算机学院

并行程序设计调研报告

协程技术调研

从并发演进到底层原理、语言对比与实验验证

作者: kyc

日期: 2026 年 6 月

目录

1	摘要	2
2	并发执行机制为何演进到协程	2
2.1	从进程、线程到事件循环	2
2.2	协程解决的不是“计算速度”，而是“等待方式”	3
3	协程的定义、分类与底层机制	3
3.1	定义与心智模型	3
3.2	有栈协程与无栈协程	4
3.3	无栈协程：编译器生成状态机	4
3.4	有栈协程：用户态上下文切换	4
4	主流语言支持对比	5
4.1	有栈协程语言：Go 与 Java 21	5
4.2	无栈协程语言：C++20、Python、C#、Rust	5
4.3	运行时与操作系统的协同	6
5	实验设计与结果	7
5.1	实验环境与方法	7
5.2	I/O 密集型结果	7
5.3	CPU 密集型与切换开销	8
5.4	内存使用对比	9
5.5	实验局限性	9
6	选型建议与总结	10
6.1	选型建议	10
6.2	总结	10
7	AI 使用说明	10

1 摘要

随着云服务、网络后端和数据处理系统的并发度不断提高，传统“一个请求一个线程”的编程方式逐渐遇到内存、调度和可维护性瓶颈。协程提供了一种比操作系统线程更轻量的并发抽象：程序员仍然书写接近同步风格的代码，而编译器和运行时在暂停点保存执行状态，并在 I/O 就绪或调度条件满足时恢复执行。

本文围绕“为什么会演进到协程、协程如何实现、主流语言如何支持、协程与线程性能差异如何”四个问题展开。结论可以概括为三点：第一，协程的核心价值不是让单段 CPU 计算更快，而是用更低的内存和调度开销承载大规模 I/O 并发；第二，协程机制需要编译器、语言运行时和操作系统 I/O 多路复用协同完成；第三，语言选型时应关注调度器、取消语义、阻塞调用兼容性和现有生态，而不是只比较“是否支持 `async/await`”。

2 并发执行机制为何演进到协程

2.1 从进程、线程到事件循环

早期服务器常用多进程模型，每个请求或连接由独立进程处理。这种方式隔离性强，但进程地址空间、页表、IPC 和上下文切换成本较高。线程共享地址空间，创建与通信成本低于进程，因此成为长期主流。但线程仍由内核抢占式调度，默认栈空间常以 MB 计；当连接数从数千增长到数万甚至百万时，线程模型会同时受到内存占用、调度队列和锁竞争限制。

事件循环模型通过非阻塞 I/O 与 `epoll`、`kqueue`、IOCP 或 `io_uring` 等机制，将大量连接复用到少量线程上 [6, 1]。它解决了线程数量过多的问题，却把程序逻辑拆成大量回调函数：局部变量、异常传播和控制流都被手工搬运到回调链中，形成所谓“回调地狱”。协程可以理解为对事件循环的抽象封装：底层仍然等待 I/O 事件，表层却恢复为线性的函数书写方式。

表 1: 典型并发模型对比

维度	进程	线程	事件循环 + 回调	协程
调度者	操作系统	操作系统	用户态事件循环	用户态调度器/运行时
切换成本	高	中	低	低
内存成本	地址空间和内核结构	默认栈较大	少量闭包对象	frame 或小栈
编程风格	IPC 明显	同步风格 + 锁	回调嵌套	同步风格 + await/yield
适合场景	强隔离任务	CPU 并行、普通并发	底层网络库	高并发 I/O、异步业务流

2.2 协程解决的不是“计算速度”，而是“等待方式”

协程的暂停点通常出现在 I/O、定时器、channel 收发或显式 yield 处。暂停时，当前协程让出执行权，运行时调度其他可运行协程；事件就绪后再恢复原协程。这一过程避免了“线程阻塞在内核中等待”，也避免了大量线程长期占用栈空间。因此协程最适合高并发 I/O，而不是直接加速 CPU 密集型计算。

3 协程的定义、分类与底层机制

3.1 定义与心智模型

协程的概念最早可追溯到 1963 年 Conway 的编译器设计 [3]，1966 年 Bernstein 对其进行了系统化阐述 [2]。协程可以被看作“可暂停和恢复的函数”。普通函数调用以后必须一路执行到返回，而协程可以在 await、yield 或阻塞点处挂起，保存局部变量、程序计数位置和必要的运行时信息，之后再由调度器恢复。

Listing 1: C++20 风格的协程伪代码

```
1 Task<Response> handle(Request req) {  
2     auto user = co_await db.query(req.uid);  
3     auto data = co_await rpc.fetch(user);  
4     co_return render(data);  
5 }
```

上面的代码表面上是顺序执行，实际上每个 co_await 都可能把控制权交还给调度器。与回调写法相比，协程把状态机生成工作交给编译器和运行时，降低了业务代码的

心智负担。

3.2 有栈协程与无栈协程

按是否拥有独立执行栈，协程可以分为有栈协程和无栈协程 [4]。

- **有栈协程：**每个协程拥有自己的用户态栈，切换时保存/恢复寄存器和栈指针。优点是可以在较深调用链中挂起，普通函数和协程函数边界更模糊；缺点是需要管理栈空间和栈增长。Go goroutine 与 Java 21 虚拟线程都接近这一类。
- **无栈协程：**协程函数被编译器改写成状态机，局部变量存放在协程 frame 中。优点是内存更小，编译器优化空间大；缺点是函数颜色问题明显，只有被标记的 `async/-coroutine` 函数才能挂起。C++20、Python `asyncio`、C# `async/await`、Rust `async` 都属于这一类。

3.3 无栈协程：编译器生成状态机

无栈协程的典型实现是编译器把函数拆成多个状态。每个挂起点对应一个状态编号，局部变量和临时值存放在 frame 中，恢复时根据状态跳转到对应位置继续执行。C# 编译器生成 `IAsyncStateMachine`，C++20 编译器生成 `coroutine frame` 和 `coroutine_handle`，Python 的 `async def` 返回 `coroutine` 对象，运行时将其包装为 `Task` 后由事件循环推进 [8]。

这种方式的优势是内存占用可控，很多协程 frame 只有几十到几百字节；切换本质上接近一次函数调用或间接跳转。但它也带来函数颜色问题：普通函数不能直接 `await`，同步 API 与异步 API 需要分别维护。

3.4 有栈协程：用户态上下文切换

有栈协程的切换通常包括保存 callee-saved 寄存器、保存旧栈指针、切换到新栈并恢复寄存器。简化后的 x86_64 上下文切换可以概括为如下形式：

Listing 2: 有栈协程上下文切换示意

```
1 switch_ctx:
2     push rbp
3     push rbx
4     push r12-r15
5     mov [rdi], rsp ; 保存旧栈指针
6     mov rsp, [rsi] ; 切换到新协程栈
7     pop r15-r12
8     pop rbx
```

```
9    pop rbp
10    ret
```

Go 的 goroutine 初始栈很小 (2KB)，并可按需增长到 GB 级；运行时通过 G-M-P 调度模型把大量 goroutine 映射到较少的操作系统线程上。Java 21 虚拟线程则把轻量线程引入 JVM 标准能力，虚拟线程运行在 carrier 平台线程之上，由 JVM 调度并在多数 JDK 阻塞 I/O 处自动让出 carrier[7]。

4 主流语言支持对比

4.1 有栈协程语言：Go 与 Java 21

Go 语言自 1.0 (2012) 起就内建了 goroutine 和 channel。goroutine 是有栈协程，初始栈仅 2KB，运行时通过 G-M-P 模型调度 [5]：G 是 goroutine，M 是操作系统线程，P 是逻辑处理器（持有本地运行队列）。网络 I/O 通过内建的 netpoller（底层使用 epoll/kqueue/IOCP）自动挂起和恢复。Go 1.14 引入基于信号的抢占式调度，避免长时间计算的 goroutine 饿死其他任务 [9]。

Java 21 (2023) 通过 JEP 444 引入虚拟线程 [7]。虚拟线程是 JVM 管理的有栈协程，运行在少量 carrier 平台线程之上。JDK 内所有阻塞 API (Socket、JDBC、Thread.sleep) 已被改造为在阻塞点自动 yield carrier，而非阻塞内核线程。虚拟线程的最大卖点是“同步代码无需修改”：开发者用 Thread.ofVirtual() 替换 new Thread() 即可获得协程级并发能力。但 synchronized 块和 JNI 调用会把虚拟线程 pin 到 carrier 上，是使用时的主要陷阱。

4.2 无栈协程语言：C++20、Python、C#、Rust

C++20 (2020) 在语言层面引入了 co_await、co_yield、co_return 三个关键字，但标准库不提供调度器、Task 类型或异步 I/O。开发者需要依赖 cppcoro、libcoro 或 Boost.Asio 等第三方库来构建完整的异步生态。这种“只给机制，不给策略”的设计给予了最大灵活性，但也提高了入门门槛。

Python 的 asyncio (PEP 492/525/530) [8] 使用单线程事件循环，async def 返回 coroutine 对象，被包装为 Task 后由事件循环推进。底层通过 selector 模块调用 epoll/kqueue 等系统调用。Python 的 GIL 限制了 CPU 密集型任务的并行能力，因此 asyncio 最适合 I/O 密集型场景。阻塞同步调用（如 time.sleep、requests.get）会阻塞整个事件循环，是使用时的主要风险。

C# 5.0 (2012) 是现代 async/await 的“开山鼻祖”。编译器把 async 方法改写为 IAsyncStateMachine，Task 类型充当 Future，SynchronizationContext 决定恢复在哪

个线程（WinForms/WPF 回 UI 线程，ASP.NET 回请求线程）。C++20、Rust、Python 的 `async/await` 在语义和实现上都深度借鉴了 C# 的设计。

Rust（1.39，2019）的 `async/await` 编译为 `Future` 状态机，但语言不提供运行时，需要 `tokio` 或 `async-std` 等第三方库。Rust 的所有权和生命周期系统使得 `async` 函数的 `frame` 管理更复杂，学习曲线较陡，但换来了零成本抽象和无垃圾回收的性能保证。

表 2: 主流语言协程能力对比

语言	关键机制	类型	调度器	主要特点
Go	<code>go/channel</code>	有栈	GMP 内建	语法轻，普通函数可直接作为 <code>goroutine</code>
Java 21	Virtual Thread	有栈	JVM ForkJoin	保持同步代码风格，降低迁移成本
C++20	<code>co_await</code>	无栈	标准不提供	只给底层机制，Task 和 I/O 需库实现
Python	<code>async/await</code>	无栈	单线程事件循环	适合 I/O；CPU 密集受 GIL 限制
C#	<code>async/await</code>	无栈	线程池/上下文	工程化成熟，UI/服务端生态完善
Rust	<code>async/await</code>	无栈	第三方运行时	零成本抽象强，但生态和学习成本高

4.3 运行时与操作系统的协同

协程不是脱离操作系统独立存在的机制。它通常需要三层协作：

1. **编译器或解释器：**识别 `async/await`、`yield` 或 `coroutine` 关键字，生成 `frame`、状态机和恢复入口。
2. **语言运行时：**维护可运行队列、挂起队列、Task/Future、定时器和调度策略；Go 还包括 `work stealing`、`netpoller` 和抢占逻辑 [9]。
3. **操作系统：**提供非阻塞 I/O、多路复用、线程和定时器等基础能力。运行时等待 `fd` 就绪后，再恢复对应协程。

5 实验设计与结果

5.1 实验环境与方法

实验在同一台本机上运行，硬件与软件环境如下：

- CPU: 13th Gen Intel(R) Core(TM) i9-13900H, 14 核 20 逻辑处理器;
- 操作系统: Windows 11 Home China 64-bit, 版本 10.0.26200;
- Python: 3.12.9; Go: 1.25.6; Java: OpenJDK 21.0.5;
- 实验日期: 2026 年 5 月 27 日 (重测); 图表由 `plot_results.py` 在 `test` 环境中生成。

测试分为三类: I/O 密集型测试创建 10000 个任务, 每个任务 `sleep 100 ms`, 用于模拟大量网络等待; CPU 密集型测试创建 1000 个任务, 每个任务计算 `fib(25)`; 上下文切换测试使用大量 `yield/sleep(0)`、`channel` 交互或轻量任务估计调度开销。每项测试重复 5 次, 报告均值和标准差。完整代码见同目录下 `bench.py`、`bench.go` 和 `Bench.java`。原始输出记录在 `results/` 目录中。

5.2 I/O 密集型结果

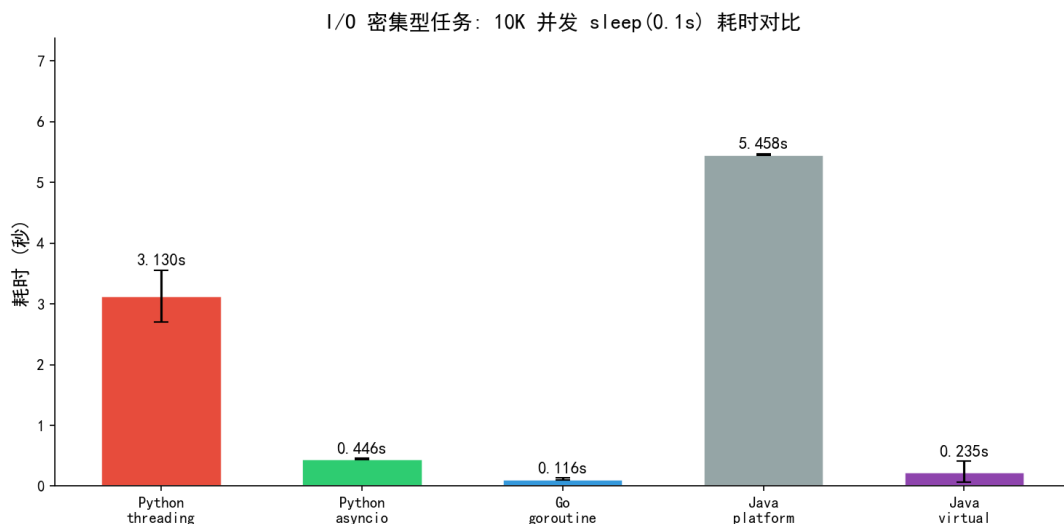


图 1: 10000 个 I/O 等待任务耗时对比 (5 次重复, 误差线为标准差)

表 3: I/O 密集型实验结果（5 次重复）

模型	10000 任务耗时	对照口径
Python threading	3.130±0.426 s	Python 线程基线
Python asyncio	0.446±0.012 s	比 Python threading 快约 7.0×
Go goroutine	0.116±0.020 s	Go 运行时内建轻量任务
Java platform thread（200 固定线程池）	5.458±0.013 s	Java 线程池基线
Java virtual thread	0.235±0.173 s	比 Java platform thread 快约 23.2×

结果符合预期：I/O 密集场景中，协程/虚拟线程能在等待期间释放执行权，使少量内核线程服务大量逻辑任务。Python asyncio 相比 Python threading 快约 7.0 倍；Go goroutine 和 Java 虚拟线程都能在 0.2 秒量级完成 10000 个 sleep 任务，Java 虚拟线程相比固定平台线程池快约 23.2 倍。Java 虚拟线程的标准差较大（±173 ms），可能与 JIT 预热有关。

5.3 CPU 密集型与切换开销

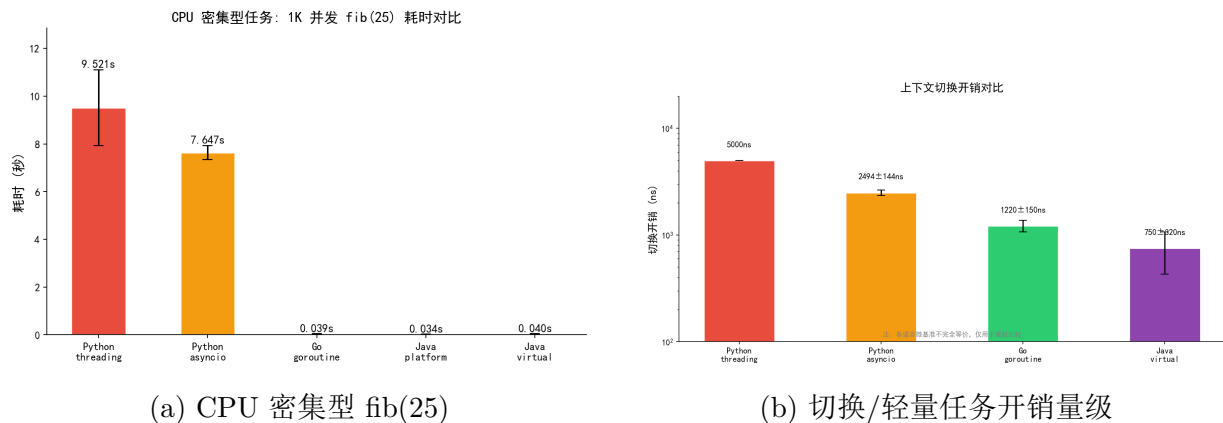


图 2: CPU 密集型和调度开销实验

Python 的 threading 与 asyncio 在 CPU 密集型测试中都没有获得真正的多核计算收益，threading 还会额外承担大量线程创建和 GIL 竞争开销；这说明协程并不会减少计算本身的 CPU 成本。Go 和 Java 的 CPU 测试更快，主要原因不是“协程让计算变快”，而是运行时可以把任务分配到多核上执行。Java 平台线程池在该测试中略快于虚拟线程，也说明虚拟线程的优势不在短小 CPU 任务，而在大量阻塞等待任务。

切换开销图使用对数坐标展示量级差异。Python asyncio 的百万次 yield 约为 2.494±0.144 秒，Go goroutine 的 100000 次 channel 交互约为 122±15 ms，Java 虚拟线程的 100000

个 `sleep(0)` 任务约为 75 ± 32 ms。由于不同语言的微基准并不完全等价，这些数字更适合用于判断量级，而不应作为绝对排名。

5.4 内存使用对比

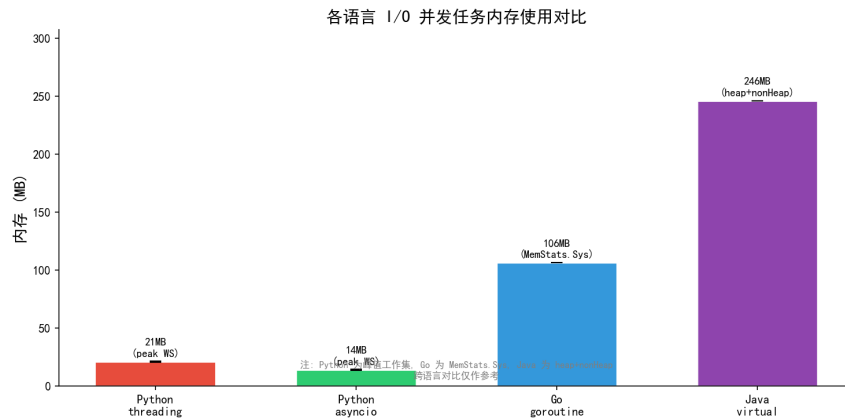


图 3: 各语言 I/O 并发任务内存使用对比 (10K 任务)

本次实验中，Python 使用采样线程记录 benchmark 期间的进程峰值工作集，Go 使用 `runtime.ReadMemStats` 的 `Sys` 字段，Java 使用 `MemoryMXBean` 的 `heap + nonHeap` 用量。三种语言使用不同的内存测量方法和运行时模型，因此跨语言对比仅作参考。Python 的内存优势主要来自协程 frame 极小（几十到几百字节）；Go/Java 的较高内存反映了运行时系统本身（调度器、GC、JIT 编译器等）的开销。

5.5 实验局限性

本次实验存在以下局限性，读者在参考数据时应注意：

- I/O 模拟：**使用 `sleep(0.1s)` 模拟网络等待，不涉及真实协议栈和网络延迟。
- CPU 测试：**`fib(25)` 用来展示“协程不自动加速计算”；Go/Java 的优势来自多核调度，不来自协程语义本身。
- 切换测试：**Python `asyncio yield`、Go `channel`、Java `sleep(0)` 的操作不完全同构，适合看数量级，不适合做精确排名。
- 内存测试：**三种语言使用不同测量方法，跨语言对比仅作参考。

6 选型建议与总结

6.1 选型建议

如果项目瓶颈在 CPU，优先考虑并行计算手段，而不是协程。如果瓶颈在 I/O 且并发量较小，传统线程池已经足够；当并发量达到数千以上时，再考虑协程或虚拟线程。新建高并发后端服务可以优先考虑 Go goroutine；JVM 旧系统希望保留同步代码风格时，Java 21 虚拟线程很有吸引力；Python Web 或爬虫任务适合 asyncio/FastAPI/uvloop，但必须严格避免阻塞调用；系统级高性能服务可以考虑 Rust tokio 或 C++20 协程与 io_uring/asio 组合。

6.2 总结

协程的本质不是“神奇的快速线程”，而是把等待态任务从昂贵的内核线程中解放出来，用编译器状态机、用户态调度器和操作系统 I/O 多路复用共同支撑高并发。它让程序员可以用近似同步的代码表达异步控制流，是云时代服务端编程的重要抽象。

从技术趋势看，协程正在朝两个方向发展：一类是 C++20/Rust/Python/C# 代表的无栈 async/await，强调精细控制和低内存；另一类是 Go goroutine 与 Java 21 虚拟线程代表的有栈轻量线程，强调普通同步代码的可扩展性。面对具体工程问题时，最关键的判断不是“哪种协程更先进”，而是瓶颈是否在 I/O、阻塞调用能否被运行时接管、错误和取消能否被结构化管理。

7 AI 使用说明

按照作业要求，本次调研和材料制作中使用 AI 的方式如下：

1. 使用 AI 辅助梳理“并发演进-协程原理-语言对比-实验验证-选型建议”的讲述结构，再人工调整顺序和重点。
2. 使用 AI 生成和校对部分示意图、表格和代码片段的表达，但实验代码在本机运行后用真实数据替换。
3. 使用 AI 帮助检查术语一致性，例如有栈/无栈协程、函数颜色、GMP 调度器、virtual thread pinning 等。
4. 对关键版本号、JEP/PEP、语言特性和运行时机制回查官方文档或源代码说明，避免把 AI 输出直接当作事实。
5. 最终 Slides 与报告文字均围绕本人实验和理解重新组织，避免大段复制资料原文。

参考文献

- [1] Jens Axboe. io_uring - linux kernel async i/o interface. https://kernel.dk/io_uring.pdf.
- [2] A. J. Bernstein. Coroutines: A programming methodology. *Technical Report OSR-66-1*, 1966.
- [3] Melvin E Conway. Design of a separable transition-diagram compiler. *Communications of the ACM*, 6(7):396–408, 1963.
- [4] Ana Lúcia de Moura and Roberto Ierusalimsky. Revisiting coroutines. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 31(2):1–31, 2009.
- [5] M. Kavya. Go scheduler. <https://www.youtube.com/watch?v=YHR05WQGh0k>, 2020.
- [6] Linux man-pages project. epoll - i/o event notification facility. <https://man7.org/linux/man-pages/man7/epoll.7.html>.
- [7] Oracle. Jep 444: Virtual threads. <https://openjdk.org/jeps/444>, 2023.
- [8] Yury Selivanov. Pep 492 – coroutines with async and await syntax. <https://peps.python.org/pep-0492/>, 2015.
- [9] The Go Authors. Go runtime source - runtime/proc.go. <https://github.com/golang/go/blob/master/src/runtime/proc.go>.